

HESTIA

Planejamento técnico do HestIA

Estratégia de execução do MVP, dependências, riscos, evidências e Definition of Done

Tarefa: SCRUM-150

Disciplina: Modelagem e Construção de Software

Épico Jira: SCRUM-99 - USD02

Sprint: Sprint 1

Responsável no Jira: Ricardo Gul D'Avila

Data-base: 2 de setembro de 2026

Status do documento: Versão de trabalho para revisão da equipe

Base de elaboração: materiais do Canvas, estudo de caso do PI-IV, cards do Jira, ADR-001 e estado observado do repositório HestIA. Os rótulos de disciplina do Jira foram tratados como metadados, não como fonte acadêmica principal.

1. Síntese executiva

Este planejamento organiza a construção do primeiro incremento demonstrável do HestIA. O produto será desenvolvido como um monólito modular em Java 21 e Spring Boot, com PostgreSQL e um componente auxiliar Node.js para alertas e notificações. A escolha preserva simplicidade operacional, mas exige limites explícitos entre os módulos de negócio.

Objetivo da Sprint 1: entregar documentação arquitetural coerente, um primeiro monólito modular executável e evidências rastreáveis no Jira e no GitHub, conforme o Entregável 1 do Canvas.

2. Entradas, escopo e premissas

2.1 Escopo funcional do MVP

- Cadastrar e consultar empresas, departamentos e usuários.
- Cadastrar e consultar ferramentas de IA vinculadas a uma empresa.
- Cadastrar, versionar e consultar políticas de uso de IA.
- Registrar dados iniciais de uso ou consumo e produzir alertas.
- Disponibilizar indicadores básicos para gestores e administradores.
- Executar um fluxo demonstrável com configuração reproduzível.

2.2 Fora do escopo imediato

- Decomposição do núcleo em microsserviços independentes.
- Alta disponibilidade, mensageria distribuída e escalabilidade automática.
- Integrações produtivas com todos os fornecedores de IA.
- Interface definitiva, design system completo ou operação em produção.
- Modelo de Machine Learning produtivo; o Entregável 1 exige definição e preparação, não produção.

3. Linha de base do repositório

O diagnóstico diferencia o que existe no código da arquitetura-alvo. Isso evita que os diagramas apresentem componentes como concluídos quando ainda são planejados.

Área	Estado observado	Ação necessária
Estrutura	Pacotes por domínio para empresa, departamento, usuário e política.	Formalizar fachadas públicas e limites entre módulos.
Ferramenta IA	Somente enum TipoFerramentaIA.	Criar entidade, contratos, serviço, repository e testes.
Política	Duas entidades PoliticaUso para a mesma tabela.	Consolidar no módulo politica e remover duplicidade.
Inicialização	Classe principal e teste de contexto não estão consolidados em develop.	Reconciliar branches e restaurar execução reproduzível.
Testes e CI	Sem testes automatizados suficientes e sem pipeline versionado.	Criar teste-base, testes dos módulos e pipeline de PR.
Segurança	Senha de usuário é armazenada e retornada sem proteção adequada.	Aplicar hash, DTO de resposta e impedir exposição.

Área	Estado observado	Ação necessária
Node.js	Componente de alertas ainda não existe.	Definir contrato, implementar e integrar após a arquitetura.

4. Direção arquitetural

- Núcleo: monólito modular Spring Boot, uma unidade principal de negócio e implantação.
- Organização: módulos de negócio com controller, DTO, fachada/serviço, domínio e repository próprios.
- Persistência: PostgreSQL compartilhado no MVP, com cada tabela sob responsabilidade de um único módulo.
- Integração Node: REST síncrono por meio de um gateway, seguindo o exemplo acadêmico de Alert Service.
- Execução: Docker Compose para API, banco e Alert Service; interface web quando o protótipo for versionado.
- Evolução: serviços separados somente quando um ASR justificar autonomia de deploy, dados ou escala.

Sequência técnica recomendada

Dependências entre as entregas do MVP

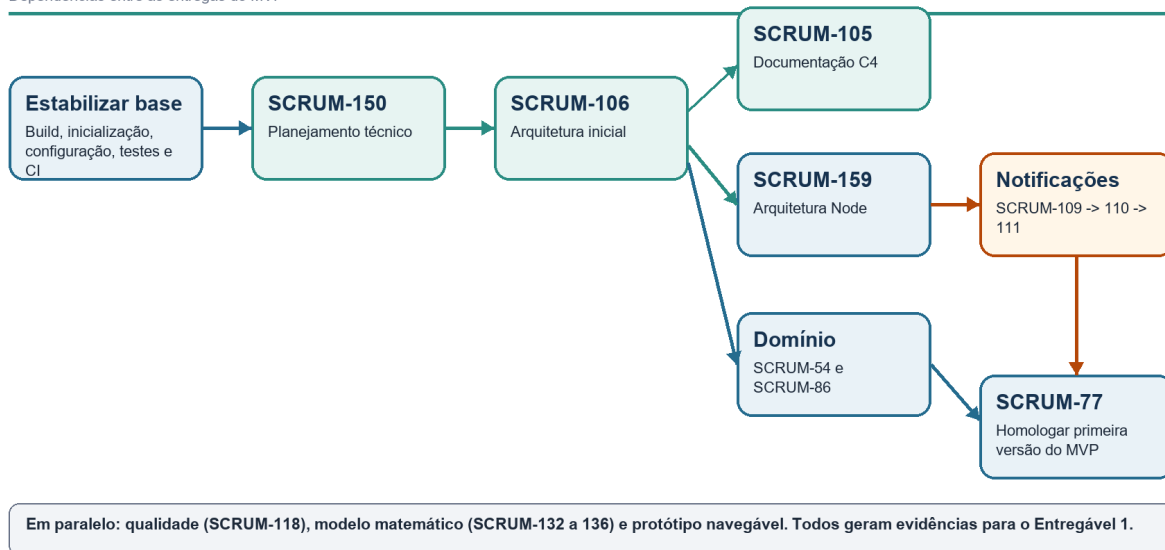


Figura 1 - Trilha e dependências técnicas do MVP.

5. Plano de trabalho

Etapa	Objetivo	Trabalho	Critério de saída
0	Estabilizar base	Build, classe principal, configuração, PostgreSQL, teste de contexto e CI.	Clone limpo testa e inicia conforme README.
1	Arquitetura	Finalizar SCRUM-150, SCRUM-106, SCRUM-105 e decisão do Node.	Documentos coerentes entre si e com ADR-001.
2	Domínio	Completar empresa e ferramenta; corrigir política e fronteiras entre módulos.	CRUDs centrais com validação e testes.
3	Alertas	Implementar Alert Service, cliente Spring e testes de contrato.	Fluxo REST reproduzível com falhas controladas.
4	Qualidade	Plano de testes, processos, métricas, CMMI e rastreabilidade.	Matriz requisito-critério-teste-evidência.
5	Homologação	Integrar incrementos, revisar documentação, protótipo e pacote do Canvas.	SCRUM-77 homologada e versão demonstrável.

6. Cronograma de curto prazo

Quando	Prioridade	Entregas verificáveis
2 set.	Arquitetura e documentação	Revisar 150/106/105; estabilizar develop; fechar decisão Node.
3 set.	Execução e evidências	Build reproduzível, correções bloqueantes, C4 final, qualidade, wireframes e testes.
4 set. até 13h30	Empacotamento	Gerar PDFs, conferir links, montar ZIP do grupo e validar submissão.
Após D1	Incremento funcional	Implementar domínio restante, Alert Service e integrações sem comprometer a base.

7. Dependências entre Scrums

Sequência	Dependência	Motivo
150 -> 106	Planejamento antes do detalhamento	A arquitetura precisa responder a escopo, riscos e prioridades.
106 -> 105	Decisão antes da representação	O C4 documenta a arquitetura decidida, não substitui a decisão.
106 -> 159 -> 109	Contrato antes do código Node	Evita implementar um serviço incompatível com o monólito modular.
54/86 -> 77	Domínio antes da homologação	O MVP agrega incrementos menores já testados.
132 -> 133 -> 134 -> 135 -> 136	Problema, modelo, cenário, validação e documento	Mantém rastreabilidade do modelo matemático.

8. Estratégia de engenharia

8.1 Branches e Pull Requests

- Criar uma branch por tarefa a partir de develop, com a chave Jira no nome.

- Usar commits pequenos e incluir a chave Jira na mensagem.
- Abrir PR para develop com objetivo, riscos, passos de teste e evidências.
- Exigir revisão humana e pipeline aprovado antes do merge.
- Usar a SCRUM-77 para homologação e Release PR, não como branch contendo todo o desenvolvimento.

8.2 Configuração e implantação

- Credenciais somente por variáveis de ambiente; versionar apenas um arquivo de exemplo.
- Health checks para API, banco e Alert Service.
- Docker Compose com dependências condicionadas à saúde dos serviços.
- README com comandos de teste, execução local e diagnóstico de erros comuns.

9. Estratégia de testes e evidências

Nível	Mínimo exigido	Evidência
Unidade	Regras de domínio e serviços sem infraestrutura.	Relatório de testes no pipeline.
Integração	Repositories, validações e contrato REST Spring-Node.	Logs e relatório automatizado.
API	Sucesso, dados inválidos, inexistentes e conflitos.	Coleção de requisições ou teste automatizado.
Arquitetura	Ausência de dependências proibidas e duplicidade de entidades.	Teste arquitetural e revisão do C4.
Aceitação	Fluxos do MVP conforme critérios do backlog.	Vídeo, captura e matriz de rastreabilidade.

10. Riscos e respostas

Risco	Impacto	Resposta
Divergência main/develop	Aplicação não executável ou perda de arquivos.	Reconciliar por PR antes de ampliar funcionalidades.
Limites apenas nominalmente modulares	Acoplamento entre repositories de módulos.	Criar fachadas públicas e teste arquitetural.
Node conflitar com ADR-001	Arquitetura incoerente.	Tratá-lo como Alert Service auxiliar e registrar decisão complementar.
Tarefas sem critérios	Conclusão subjetiva e retrabalho.	Adicionar critérios verificáveis e evidências a cada card.
Senha e dados sensíveis	Falha grave de segurança.	Hash, DTOs de resposta e revisão bloqueante.
Prazo curto do Canvas	Documentos ou build incompletos.	Priorizar artefatos obrigatórios e base executável antes de expansão.

11. Definition of Done

- Critérios de aceite descritos e atendidos.
- Código respeita fronteiras de módulo e contratos públicos.

- Testes relevantes criados e executados; pipeline aprovado.
- Configuração, contrato e decisões atualizados na documentação.
- Nenhum segredo, senha em texto aberto ou artefato local foi versionado.
- Branch, commits, PR e evidências contêm a chave Jira.
- Alteração revisada por outra pessoa e demonstrável em clone limpo.

12. Critérios de aceite da SCRUM-150

1. O documento apresenta escopo, premissas, arquitetura, etapas, dependências, riscos e Definition of Done.
2. A sequência é compatível com o Entregável 1 do Canvas e com a ADR-001.
3. O planejamento distingue claramente estado atual, arquitetura-alvo e trabalho futuro.
4. As integrações Spring Boot, PostgreSQL e Node.js possuem ordem e critérios de saída.
5. A equipe revisou responsáveis, prazos e evidências antes de publicar no Jira ou GitHub.

Referências e fontes consultadas

- Canvas ESPM. Deliverables 1 (Entregáveis 1) - Todas as Disciplinas. Curso GTECHSP-TI4A-2592, atividade 293213.
- LELES, Andréia Damasio de. Introdução à Arquitetura de Software - C4_v1. Material da disciplina Modelagem e Construção de Software, 2026.
- LELES, Andréia Damasio de. Monólitos em Camadas x Monólito Modular x Microservices. Material da disciplina, 2026.
- LELES, Andréia Damasio de. Monólitos em camadas e microservices: integração com Node via RestClient. Material da disciplina, 2026.
- ESPM. PI-IV - Projeto Estudo de Caso v2. Projeto Integrado IV, 2026.
- HestIA. ADR-001 - Adotar monólito modular com camadas internas por módulo de negócio. Status: aceita, 30 ago. 2026.
- Jira Projeto_SM. SCRUM-105, SCRUM-106 e SCRUM-150. Consultado em 2 set. 2026.
- Repositório local HestIA. Snapshot técnico de origin/develop e branch local de documentação, consultado em 2 set. 2026.
- BROWN, Simon. The C4 Model for Visualising Software Architecture. c4model.com.